For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Generating IDs @ Scale

We want to generate ids that can be used as ***primary keys*** with various entities — messages, documents, …

Desirable Properties

1 — Universally Unique: No two ids must be the same.

Typically, a primary key doesn't really have to be universally unique.
As long as it is unique across all shards within the table.
It is okay for two different tables to share the same keys.
But given a table, all the rows within it (irrespective of which shard they fall in) must have different primary keys (ids) [this is MANDATORY, not just desirable]
*This means that typically, IDs are NOT universally unique. They're unique within the DB and entity (table). But different tables can have the same primary key (because **product id 5** is clearly different from **user id 5**) and different databases (not shards, databases) can have the same primary key (because **facebook user id 5** is clearly different from **Scaler user id 5**)*

Since we might use these ids as primary keys, they will be used to identify the rows/pieces-of-data in the database

Why?

If two ids collide within the database and within the table, we will not be able to read/update the appropriate data in the database.

What about ids on different servers/entities? Can they be the same?

In a distributed system, your database can be sharded. You still want to be able to uniquely identify a document across all those shards.

- **unique:** ids should be unique both within the server and across servers (mandatory)

It is mandatory that there's no collision b/w the primary keys of different entities (users & messages)?

```
user_id: 1, 2, 3, 4, 8, 9, …
message_id: 1, 2, 3, 4, …
```

Not an issue — we don't need uniqueness across entities (user/message) or across databases (facebook/scaler)

Universally Unique: no two ids will be the same irrespective of how they're being used — unique even across all entities, and all servers, and across all databases. *(good to have)*

Irrespective of who is generating an id, when, or where, or why, … if the IDs are always unique, then they're universally unique.

2 — **Strictly Increasing:** values must only increase

a.k.a. Lexicographically Sortable

Examples

- `1, 2, 3, 4, 5, 6, 7, 8, 9, 10`
  Yes — strictly increasing & continuous (no skipping allowed)
- **`1, 3, 5, 7, 1239, 1240, 2000, 5000`**
  Yes — strictly increasing & non-continuous (skipping is allowed)
- `1, 2, 2, 2, 4, 5, 6, 7, 8, 9`
  No — monotonically increasing / non-decreasing (either increase, or stay the same)
- `1, 2, 3, 4, 5, 4, 3, 5, 9, 10`
  None of the above (neither increasing, nor decreasing)

The sequence of ids should increase (continuity is not needed strictly)

Why?

Increasing values of primary key ensure that the primary key index is append-only – faster

SQL databases use B+Trees to store indexes. Primary keys always have indexes on them (sometimes it is clustered)
Values inside the B+Tree must remain sorted..

Imagine that you've a B+Tree with the values `1, 2, 3, … 10`

- If you insert `11` this insert will happen at the "end" of the tree — rightmost leaf node
  - corresponds to an **append (sequential write)** somewhere within the existing nodes on the left
- if your key is not increasing, then you will have to "insert" in the middle of the B+ tree instead of appending at the end.
  - corresponds to a random write on the disk

Sequential writes (appends) are 10,000x faster than random writes!

Strictly increasing nature is not mandatory – but it's good to have, since it will improve our database inserts

Roughly Sortable

Most of the time the primary keys (ids) must increase strictly – but if they decrease sometime, that's not an issue *(as long as there's no collisions).*

Roughly Sortable / Roughly increasing is good enough.

3 — **Random Sparse:** only a few values should be valid & it is difficult to guess which is valid

- **Dense ids**
  - `1, 2, 3, 4, 5, 6, 7, 8, 9, …`
    (every value is valid)
  - `1, 3, 5, 7, 9, 11, 13, 15, 17 …`
    (every 2nd value is valid)

- **Sparse ids**
  - `1, 1032, 2119, 5360, 7982, 8133, …`
    - sparse & random
    - desired
  - `1, 1001, 2001, 3001, 4001, 5001, …`
    - sparse but not random
    - easy to guess
    - not desired

An id is valid if there's a value in the database with this id

Why?

Dense ids are easy to guess – if you know one id, you can change it a little bit and find out other ids.

Example 1

Consider a (made up) portal for checking your Aadhar Card details
`https://aadhar.gov.in/details?id=0123-4567-8901`

Enumeration Attack: Download details of all Indians by simply changing the aadhar-id in the URL again and again

If the ids are dense – most of the ids the attacker tries will be successful. The attacker will succeed in "enumerating" all aadhar ids of all Indians.

Will the attack work?
Number of possible aadhar ids = 10^{12}
`[0-9][0-9][0-9][0-9]` — `[0-9][0-9][0-9][0-9]` — `[0-9][0-9][0-9][0-9]`

Population of India = $1.5 * 10^9$
Sparsity = $(1.5 * 10^9) / 10^{12} = 0.15\%$

If an attacker tries out a random aadhar number (or one slightly different from an existing aadhar number), the chances that this value will be a valid aadhar is only `0.15%`.
`99.85%` of the ids the attacker tries will be invalid!
The attack won't work, because Aadhar ids are sparse.

Example 2

Given the URL of a valid youtube video, try to find other valid URLs by modifying the URL a little (or by generating a completely random, new URL)

I wrote the code for it, and tried thousands of different modifications - didn't find any valid one after almost 6500 iterations!

```
index  -  code: URL
6437  -  404: https://www.youtube.com/watch?v=xvFZIo5PgG0
6438  -  404: https://www.youtube.com/watch?v=xvxZjo5PgG0
6439  -  404: https://www.youtube.com/watch?v=xvFZjN5PgG0
6440  -  404: https://www.youtube.com/watch?v=xvFZjo5XgG0
6441  -  404: https://www.youtube.com/watch?v=xvFZjoGPgG0
6442  -  404: https://www.youtube.com/watch?v=x3FZjo5PgG0
6443  -  404: https://www.youtube.com/watch?v=xvFZjo5Pg20
6444  -  400: https://www.youtube.com/watch?v=xvFZjo5P4G0
6445  -  404: https://www.youtube.com/watch?v=fvFZjo5PgG0
6446  -  404: https://www.youtube.com/watch?v=xvFZjo5PgG5
6447  -  404: https://www.youtube.com/watch?v=xvFPjo5PgG0
easter egg: find the original URL that I started with
```

Sparse ids are hard to guess – they prevent enumeration attacks.

This is why people say that you should **never expose the database primary key** to the users. Primary keys are typically `auto-increment integers` in a SQL db.

1. They are dense — enumeration attacks are possible
2. They might change due to database migrations — existing URLs on the web pointing to old ids might break *(like the Gangnam style overflow!)*

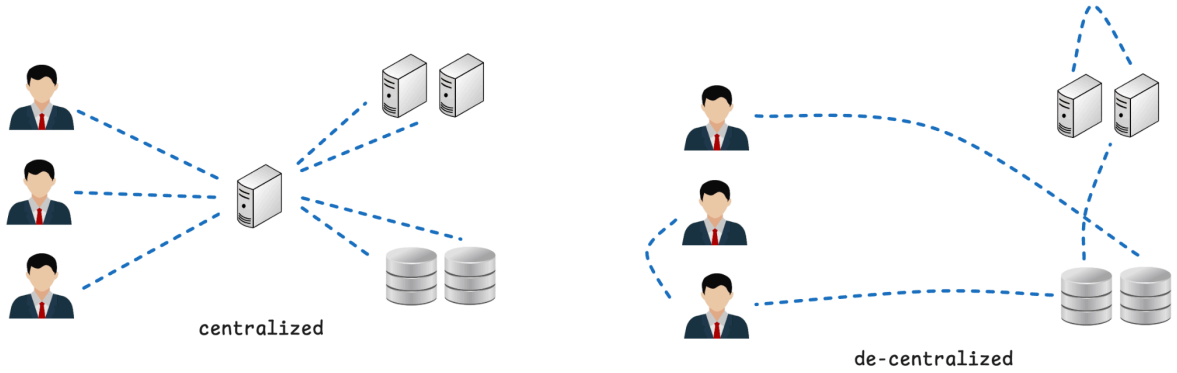
4 — **Decentralized:** anyone (even clients!) can generate ids independently

Having centralized id generation causes

- **Single Point of Failure (SPoF):** If the id generation server crashes, then we won't be able to generate ids.
- **Bottleneck:** if you want to generate million ids / second, then a single server will not be able to handle it
  Note that we don't want to lose the ids — we will have to store data on the disk — which makes it even slower, and more likely to become a bottleneck

- **High Latency:** extra network latency because you're dependent on an external entity to generate the ids for you

Centralized id generation: The clients (users/servers/dbs) are dependent on the backend server to generate the id
Decentralized id generation: Anyone (clients, backend servers, ..) can generate ids completely independently, without talking to anyone else!



Decentralized — anyone can generate ids independently — without talking to each other!

Strict?

Not strictly required, but good to have — especially for large scale distributed systems

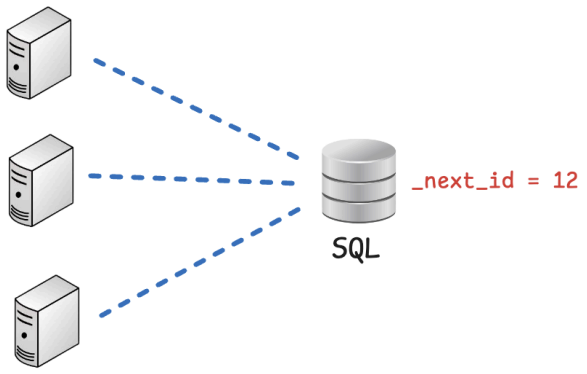
Note: *uniqueness is mandatory. Others (increasing, random sparsity, decentralized) are good to have.*

Autoincrement (counter) **in SQL**

~~Single SQL Server~~

Ensure strict uniqueness & strictly increasing nature – by incrementing the counter within a lock (*to prevent* ***race conditions***)

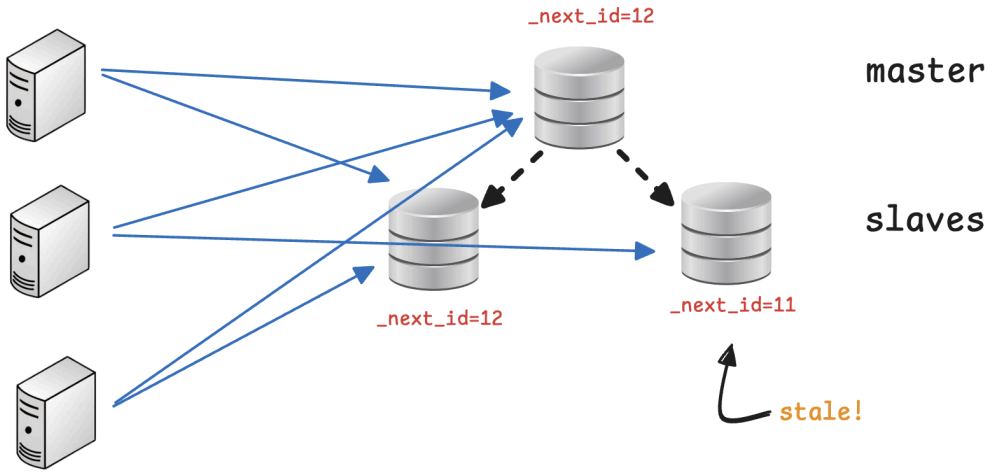
Counter — strictly increasing, guaranteed to be unique
Sparsity: failed!
Decentralized: failed!



This will be a SPoF, and a bottleneck!

~~Replication~~

Multiple SQL servers storing the same counter.
For the ids to be unique, the replicas have to be kept in-sync

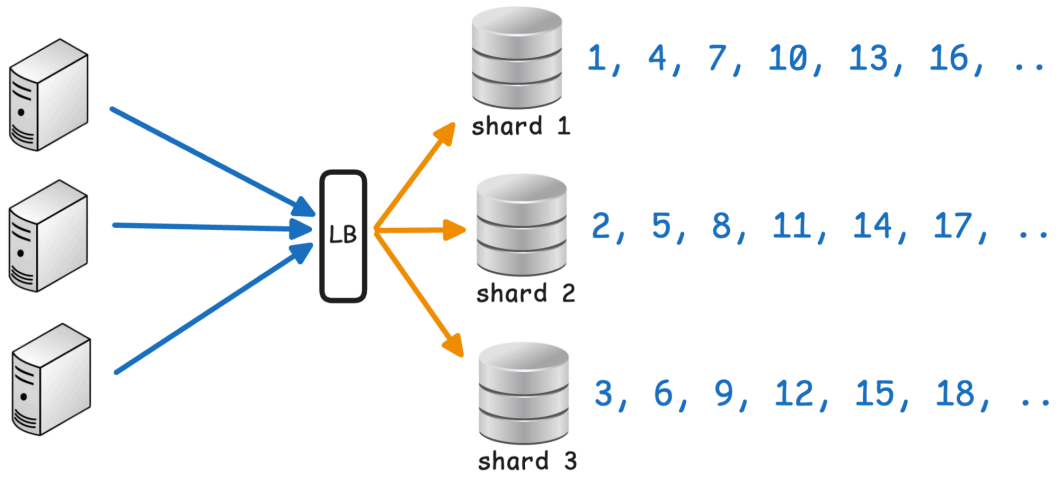


Consistency requirement: immediate consistency — because any stale read will lose uniqueness

- ~~**Write <= 2**~~**:** eventual consistency
- ~~**Write everywhere**~~**:** immediate consistency
    - bottleneck: every write must happen everywhere
    - single point of failure: even if one server is down, the writes won't succeed
    - very high latency: 2 phase commit
- ~~**Quorum**~~**:** immediate consistency
    - less of a bottleneck, but it still is – because you've to write to multiple servers
    - less available – although not a single point of failure
    - very high latency: 2 phase commit

~~Sharding – will work, but not the best solution~~

Imagine if we shard the id generation into 3 shards.
Shard `i` will generate ids of the format `3n + i`



This ensure that there are no collisions!

- **Uniqueness:** check
- **Single Point of Failure:** no issues
- **Latency:** small (still there, because the clients have to talk to the server)
- **Decentralized:** failed! (still centralized) — the clients still depend on these backend SQL servers to generate the ids
- **Sparse:** difficult to handle but possible
- **Increasing:** within the same server, yes! Any B+Tree index is only within 1 server, so this is good enough

The reason why this is still not perfect

1. centralized
2. we still want “durability” and “fault tolerance”
    - we will have to store the counter on disk — slow
    - we will need to replicate — consistency issues & high latency
3. if the number of shards change, we will have to modify the logic to ensure uniqueness for any future values

Unique — UUIDv4

What's the simplest thing to do, if we just want uniqueness? (ignore other properties)

- We can have an auto-increment counter
  - already saw that this has issues
  - requires setup of a SQL server (not the simplest thing to do)
- Just use a UUID v4

UUIDv4 - 128 bit random string

binary representation:
010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100-010110100
hex representation: af32bcd…

Note: representation (binary/hex) doesn't matter – the value is just a bunch of random binary data

If we generate multiple random values, then they're very likely to be different from each other.

Does it really guarantee uniqueness?

Any collision (even a single one) will mean that we lose data!

UUID does not guarantee uniqueness! It is totally possible (by sheer chance) for two random bitstrings to be the same.

We don't care — because the probability of collisions is so low that it will never happen practically.

" If we generate 1 billion uuids every second, it will take 86 years for a 50% chance of collision "

UUIDv4 gives us

- **uniqueness:** yes! Universal uniqueness (across all entities & servers) — *highest possible level of uniqueness*
- **decentralized:** anyone can generate ids anywhere for any reason without telling anyone; it will still be unique! — *highest possible level of decentralization*
- **random sparsity:** There's $2^{128} = 3 * 10^{38}$ possible values; very very very sparse — *highest possible level of random sparsity.*
- **strictly increasing:** Big FAIL !! Because it is random by nature, it is NOT strictly increasing

Strictly Increasing — Time

What's the simplest thing to do, if we just want strictly increasing**? (ignore other properties)**

- We can have an auto-increment counter
    - already saw that this has issues
    - requires setup of a SQL server (not the simplest thing to do)
- **Just use time!**

In our universe Time is always increasing – never stays the same, never decreases!

Does it really guarantee monotonicity?

Not really! In computers, it's kinda messy

1 - Network Time Protocol (NTP)
Used to synchronize clocks across servers by exchanging network packets and timing them

Computers maintain time by using oscillating crystals — unfortunately, there's always some defect in the crystal lattice — no two oscillators are the same.

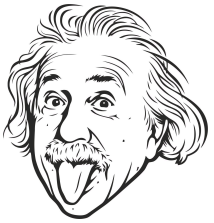
So the clocks inside different computers will run at slightly different frequencies — over time they will get out of sync (they will ***drift*** apart)

NTP — by exchanging packets it will try to figure out how to sync the clocks

- **if the clocks are out of sync by < 25 ms**
    - NTP will literally either slow down your CPU clock frequency
    - (very rare circumstances) it might turn the clock back!
- **if the clocks are out of sync by > 25 ms**
    - it will just give up and wait for developer intervention

Unfortunately in computers, the time is not strictly increasing 🙁

2 - General Relativity
Due to time dilation, it is impossible (not hard, literally impossible) in our universe to perfectly sync clocks!
Time is relative!
If you're in India vs in Siberia, the time will move at different speeds (measurably slower)



Time is "mostly" increasing in computers; not strictly!

That is okay, because we anyway didn't need strict increasing nature – we just want roughly sortable.

Resources (optional)

- **Network Time Protocol**
  - Network Time Protocol - Wikipedia
  - Internet time synchronization: the network time protocol - Communications, IEEE Transactions on
- **Time Dilation**
  - What is Time Dilation and How Does it Affect GPS?
  - Astronomy - General Relativity (14 of 17) Proof of Theory: Gravitational …
  - Impossible Muons

Idea — what if we merged the two !!? (uuid+time)

Twitter's Snowflake

Situation

- It's `2010`
- Twitter started with a small SQL database on a single server — and quickly grew out of it.
- They decided to shard the SQL db wherever possible
- if not, move the data to Cassandra.
- Cassandra has no built-in way of generating unique ids
  - Cassandra is distributed — it is not supposed to generate ids in a distributed setting
  - similarly, a sharded SQL database provides nothing to generate unique ids across servers
    - sql does not even support sharding by default
  - *`timeuuid` was added in v2.1 in 2014*
- needed tens of thousands of ids per second
- existing ids used in SQL were `64 bits`
  - ~~update all codebases (100,000+) to use 128 bits~~
    - nightmare! There will always be some bugs
  - find a solution that fits in 64 bits

Solution

Sign Bit	Timestamp	Datacenter ID	Machine ID	Sequence No.
1 bit	41 bits	5 bits	5 bits	12 bits
0	1-41	42-46	47-51	52-63

- **Sign Bit:** reserved for future use
  - Why? They were using Java, and java does not have unsigned datatypes! And they didn't want their ids to be `-ve`
- **Timestamp:** number of milliseconds passed since `4 Nov 2010`
  - typically when you store time it is measured as the number of milliseconds that have passed since `1 Jan 1970` (UTC timestamp)
  - reason for changing is that measuring from 1 Jan 1970 requires 64 bits of data
  - Twitter only had 41 bits — so they had to measure from a later date to reduce the size of the timestamp value
- **Datacenter ID / Machine ID**
  - `32` possible datacenters,
  - `32` possible servers in each datacenter
  - for a total of `1024` total possible cassandra servers
  - ensures that ids generated on different servers are different — no id collision across servers
- **Sequence number:** for monotonicity & uniqueness within same millisecond

```
datacenter_id = 12
machine_id = 10

time_start_ms = time('4 Nov 2010')

seq_no = 0
fn generate_id():
    timestamp_ms = time.now_ms() - time_start_ms
    return seq_no                 # 12 bits
           | machine_id << 12     # 5 bits
           | datacenter_id << 17  # 5 bits
           | timestamp_ms << 22   # 41 bits
         # | 0 << 63              # sign bit is always 0
```

Sequence Number?

What if you need to generate multiple ids within the same server in a tight loop?

```
fn notify_subscribers():
    for each subscriber of Scaler:
        msg_id = generate_id()
        send({ id: msg_id, content: "1% better everyday!"})
```

In this case, since the function `generate_id()` is being called multiple times within the same millisecond, the timestamp will remain unchanged — so the same id will be generated again and again in the loop!

Solution
If the millisecond hasn't changed b/w two calls, update the sequence number (counter). This will ensure that even if we call `generate_id()` multiple times within the same millisecond, the ids remain unique and increasing.

```
datacenter_id = 12
machine_id = 10
time_start_ms = time('4 Nov 2010')

prev_timestamp_ms = 0
seq_no = 0

fn generate_id():
    timestamp_ms = time.now_ms() - time_start_ms

    with lock():  # for tread safety
        if prev_timestamp_ms == timestamp_ms:
            # the millisecond has not changed
            seq_no++
        else:
            seq_no = 0
    prev_timestamp_ms = timestamp_ms

    return seq_no                  # 12 bits
           | machine_id << 12      # 5 bits
           | datacenter_id << 17   # 5 bits
           | timestamp_ms << 22    # 41 bits
         # | 0 << 63               # sign bit is always 0
```

Why can't we just use timestamp in nanoseconds, instead of sequence number?

[Java Docs - **`currentTimeMillis()`** vs **`nanoTime()`**]

Every programming language always provides 2 functions to get the current time

1. **millisecond accuracy:** this is what we typically use to find the current time
    a. allows you to find what the current time is (to millisecond accuracy)
2. **nanosecond precision:** doesn't give you accuracy, only precision — the time can be off by a significant value (the difference will be accurate)
    a. it allows you to measure the "elapsed time" between two events

```
start = nanoTime()
// do some complex calculation
end = nanoTime()
print("It took", end - start, "nanoseconds)
```

Basically, the CPU clock is running at nanosecond frequency, so you can measure "elapsed" time at nanosecond level.
But, your CPU clock is only synced with other clocks at millisecond intervals, so you can only measure "clock" time (current time) at millisecond scale.

Why `4th Nov 2010`?

Absolutely no significance – they just deployed Snowflake on this date so they started measuring time from that date

Note: Twitter Snowflake is old — do not use it in production. There are better alternatives ...

ULID / UUIDv6

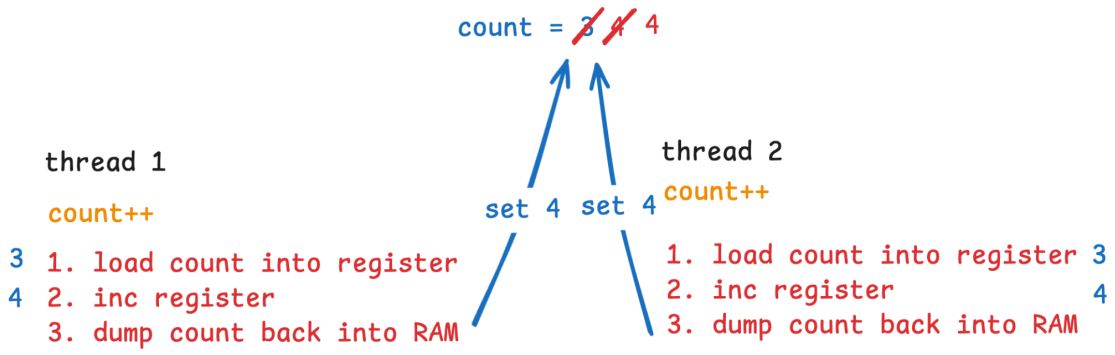
- **universally unique, decentralized, sparse, roughly sortable**
- widely accepted spec
- popular implementations in every major language
- 128 bit
- high performance

Resources (optional)

- **Custom Implementations**
  - **Twitter —** Announcing Snowflake
  - **Sony —** sony/sonyflake: A distributed unique ID generator inspired by Twitter's Snowflake
- **ULID:** The canonical spec for ulid
- **UUIDv6**
  - New UUID Formats
  - Draft Prototypes and Tests for UUIDv6 and beyond

Doubts

Race Condition



MD5 is cracked

It is now possible for an attacker to easily (less than a second!) generate a malicious file that hashes to the exact same hash as any given file!

For storing passwords, use BCrypt13 / …
AES 512 / 1024 is considered very secure (until quantum computers arrive)

AES 256 is not that secure – random people will not be able to crack it, but governments with millions of $$$ probably can!

How to store a UUID/ULID in database?

Most modern databases (postgres/mysql/all no-sql) have first class support..
A UUID/ULID will only take 128 bits (just 2x of the usual integer ids)

SQLite does NOT have support — you have to store it as a string
You should never use sqlite in the backend. SQLite is perfect for frontend & embedded.

NEVER use UUID(v1-v5) as primary key!

because they're random – they're not increasing.
This will cause almost a 2x – 10x slowdown in your write throughput

Use **ULID/UUIDv6** or use auto-increment ids

UUID is the datatype (128 bit string)
ULID / UUID v1-v6 are algorithms to generate the UUIDs

Python is almost 100x slower than C++

CPU frequency is typically 0.2ns
CPU is 2.3 GHz ⇒ clock frequency is 2.3Ghz ⇒ clock ticks once every 1/(2.3 billion) seconds … or, once every **0.4ns**
In every clock tick, your CPU can potentially add two 64 bit numbers

```
  1000101010001010100010101000101010001010100010101000101010001010
+ 0100010101000101010001010100010101000101010001010100010101000101
```

Pragy's fav programming language(s)

Python / Typescript / Rust / Haskell

Elegance

- Mathematical: functional programming, ADTs, monads, Y-combinators, currying, higher order functions..
- Implementation: consistency in naming, consistency in standard library design
- Ease of Use: you don't have to be an expert in the language to enjoy the language, but if you're an expert, you will enjoy it further
- Syntactic Sugar: allowing the developer "quality of life" features

Why Java Sucks

```
import java.io.*;
import java.lang.*;
public class HelloWorld {
   public static void main(String args[]) throws IOException {
      System.out.println("Hello, World!");
   }
}
```

```
print('Hello, World!)
```

```
console.print('Hello, World!')
```

```
fn main() {
   println!('Hello, World')
}
```

```
BigInt(5) + BigInt(10)
```

new BigInteger(5).add(new BigInteger(10)

SI = P * (1 + r) * t

price.multiply(new Currency(1).add(rate)).multiply(time)

Python is on average ~100 to 1000x slower than C
Numpy is written in C, and gives you massive speedups in Python
Numba / Cython / Jit (as of Python 3.13), …

CPython (the default & official intepretter) is written in C
Jython (Java)
IronPython (.Net)
RustPy (rust)
Brython (js)
Pyiodide (WASM)

How do I choose the correct programming language?

Depends on the purpose

1. **Fun:** Python / Typescript / Rust
2. **Job:** Python / Javascript (+Typescript) / Java
3. **Building a company:** Python / Typescript / Java / Ruby / Golang
4. **Go crazy (in a good way, and have braingasms):** Haskell, Rust, .. other esoteric languages